SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4307

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: **Total Marks: 30**

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

- Identify appropriate **HTML5 semantic elements** missing in the structure.
- Modify the structure to make it **standards-compliant**.
- Interpret the **HTTP request generated** during form submission.
- Analyze the **HTTP response cycle** from server to browser.
- Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

Identify **improper nesting and missing closing tags**.
Analyze how different browsers may interpret this structure.
Identify **missing semantic elements** (e.g., header, section).
Provide a **corrected W3C-compliant HTML structure**.
Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

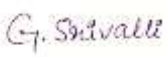
Questions:

- Identify issues in the **form configuration**.
- Analyze why the **HTTP request is not triggered**.
- Interpret the role of **GET method in this scenario**.
- Propose a **corrected implementation**.
- Suggest improvements for **secure data transmission**.

Code of Conduct:

I _G. Srivalli (192365104) _ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weig htage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconception s	Poor understandin g; major misconceptio ns
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

https://docs.google.com/spreadsheets/d/1YxbIZd9p1Dt05a1GKf6_1khTIAGSW9_Ytb0oXPkEgnk/edit?usp=sharing

Question 1: Online Symposium Portal

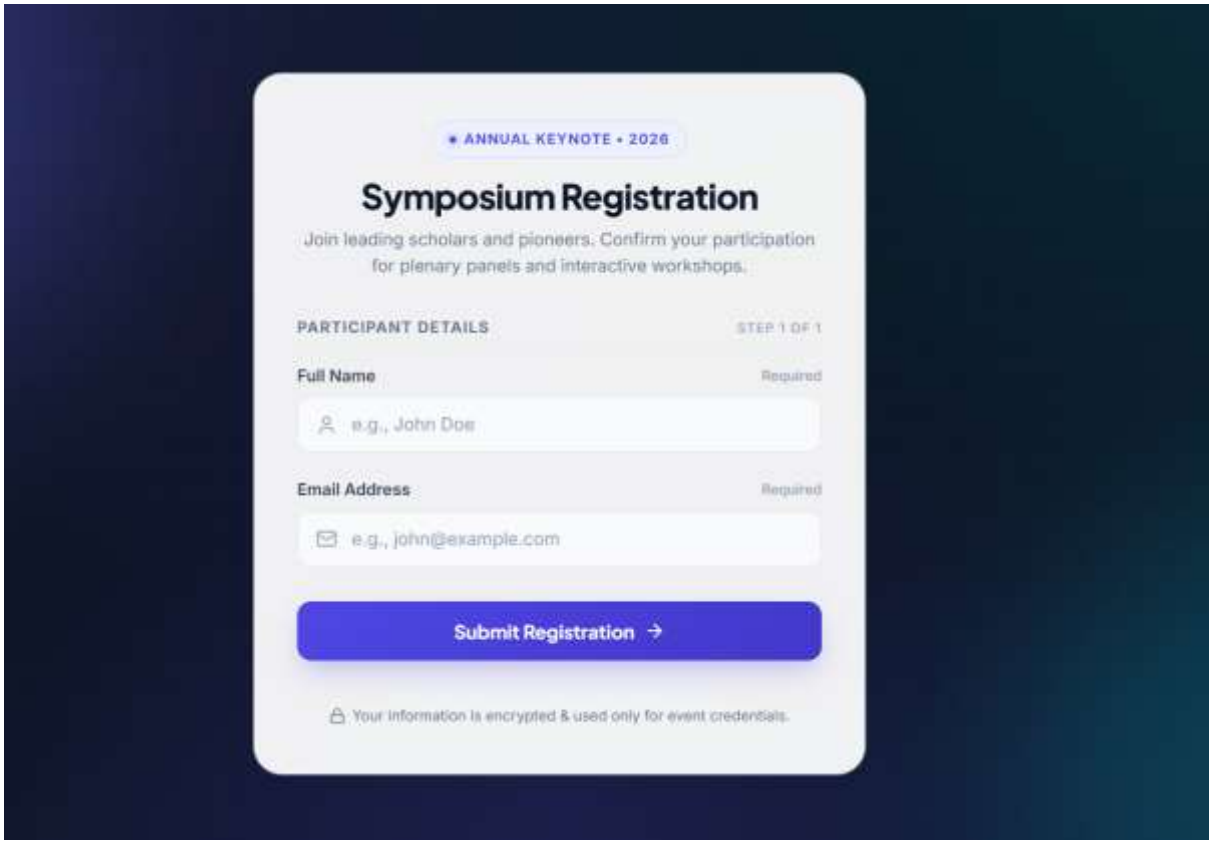
1. Identify Appropriate HTML5 Semantic Elements Missing in the Structure

- **<main> or <section>:** The initial snippet lacks structural container tags to properly designate the core context or segment of the registration interface.
- **<fieldset> and <legend>:** These tags are essential to cluster related form elements and assign a contextual accessibility caption (e.g., "Registration Details").
- **<label>:** Input fields lack explicitly bound text labels, negatively impacting usability and screen reader accessibility.
- **<header>:** An introductory header containing a clear topic heading (e.g., <h2>) should precede the interactive form content.

2. Modify the Structure to Make It Standards-Compliant

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Symposium Registration</title>
</head>
<body>
  <main>
    <section>
      <header>
        <h2>Online Symposium Registration</h2>
      </header>
      <form action="/register" method="POST">
        <fieldset>
          <legend>Student Information</legend>
```

```
        <div>
          <label for="fullName">Full Name:</label>
          <input type="text" id="fullName" name="name" required
autocomplete="name">
        </div>
        <div>
          <label for="studentEmail">Email Address:</label>
          <input type="email" id="studentEmail" name="email" required
autocomplete="email">
        </div>
        <div>
          <button type="submit">Submit Registration</button>
        </div>
      </fieldset>
    </form>
  </section>
 </main>
</body>
</html>
```



3. Interpret the HTTP Request Generated During Form Submission

When using the method="POST" configuration:

- **Request Line:** Contains POST /register HTTP/1.1.
- **Headers:**
  - Host: college.edu
  - Content-Type: application/x-www-form-urlencoded
  - Content-Length: <length_in_bytes>
- **Request Body:** Data key-value pairs are URL-encoded and transmitted securely inside the message payload rather than appended to the URL string:

```
POST /register HTTP/1.1
Host: college.edu
Content-Type: application/x-www-form-urlencoded
Content-Length: 38
name=John+Doe&email=johndoe%40college.edu
```

4. Analyze the HTTP Response Cycle from Server to Browser

Phase	Description & Status Codes
1. TCP & Logic Processing	The server accepts the incoming connection, parses HTTP headers/body payload, and triggers business application logic.
2. Server Response Phase	**Success:** Returns 200 OK with content, or 303 See Other (Post/Redirect/Get pattern) to prevent resubmission. **Validation Error:** Returns 400 Bad Request or 422 Unprocessable Entity with details. **Server Error:** Returns 500 Internal Server Error on processing failure.
3. Browser Rendering	The browser reads response headers, terminates or pools the TCP socket, builds the DOM/CSSOM trees, and presents the output.

5. Suggest Improvements for Usability and Accessibility

- **Explicit <label> association:** Link tags using for="..." matching the input target id to increase touch target space and assist accessibility tools.
- **Client-Side Validation:** Apply native attributes such as required, correct type, and pattern constraints.
- **Autofill Attributes:** Integrate autocomplete="name" and autocomplete="email" to speed up entry across devices.
- **Button Semantics:** Explicitly define type="submit" to avoid default control ambiguity.

Question 2: Cross-Browser Compatibility Issue

1. Identify Improper Nesting and Missing Closing Tags

- **Unclosed Heading:** <h1>Welcome lacks its closing </h1> partner tag.
- **Unclosed Paragraph:** About us lacks its required closing tag.
- **Improper Nesting:** The element resides inside the unclosed <h1>, creating invalid block-level nesting logic inside a heading.
- **Missing Document Type:** Lacks the mandatory <!DOCTYPE html> statement, triggering browser Quirks Mode.

2. Analyze How Different Browsers May Interpret This Structure

- **Quirks Mode Fallback:** Lacking a modern doctype forces engines (Blink, Gecko, WebKit) to revert to backward-compatibility behavior, creating layout, font rendering, and box-model variations.
- **Parser Divergence:** Modern HTML5 parsers implement automatic repair routines, but older or non-standard engines may prematurely terminate <h1> or render the directly inside the heading, leading to layout shifts and typography issues.

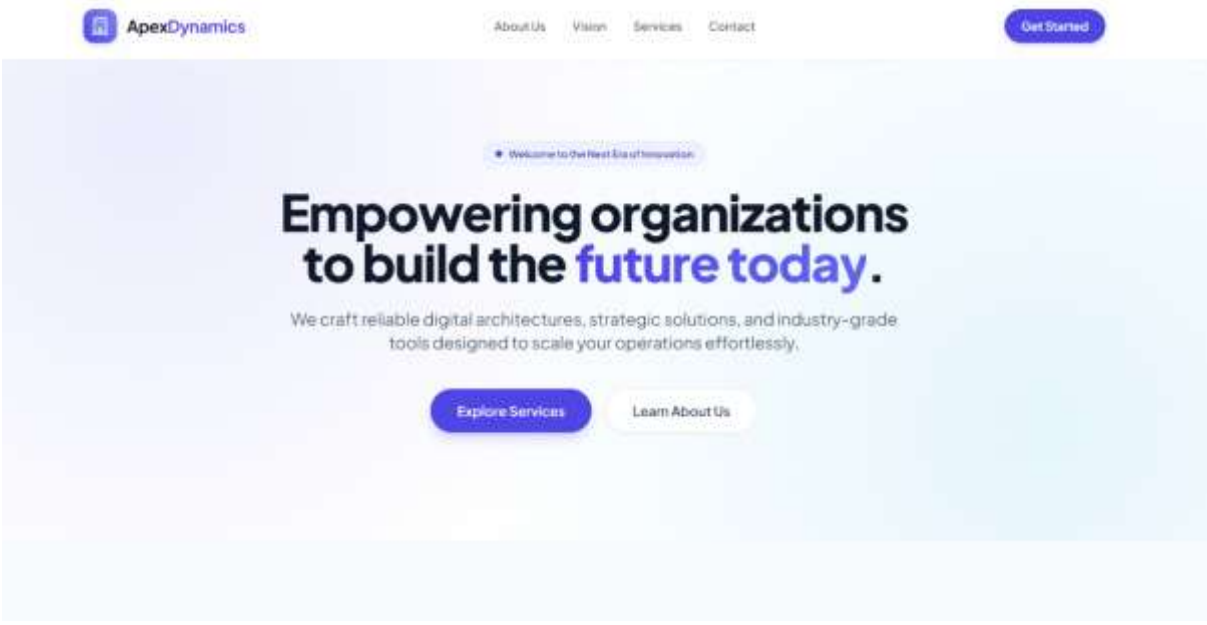
3. Identify Missing Semantic Elements

- <!DOCTYPE html> declaration to enforce standards-mode parsing.
- <meta charset="UTF-8"> character encoding specification.
- <header> to group primary site headings.
- <section> or <article> to denote meaningful context segments instead of plain nodes.
- <main> to mark unique page core contents.

4. Provide a Corrected W3C-Compliant HTML Structure

<!DOCTYPE html>

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Company</title>
</head>
<body>
  <main>
    <header>
      <h1>Welcome</h1>
    </header>
    <section>
      <p>About us</p>
    </section>
  </main>
</body>
</html>
```



5. Suggest Techniques to Ensure Cross-Browser Compatibility

- **Include <!DOCTYPE html>:** Guarantees standard document parsing rules across engine implementations.
- **Use a CSS Reset / Normalize:** Integrate normalize.css to standardize browser-default CSS styles.
- **W3C HTML Validation:** Validate source markup using automated W3C Validation Services.
- **Automated Cross-Browser Testing:** Conduct automated UI test pipelines with tools like Playwright or Selenium across Chromium, Gecko, and WebKit.
- **Static Analysis Linters:** Employ code linters (e.g., HTMLHint) within automated deployment flows to catch invalid nesting early.

Question 3: Form Submission Failure

1. Identify Issues in the Form Configuration

- **Button Type:** The action button uses type="button", defining a generic non-submitting control element.
- **Inappropriate HTTP Method:** Login details are sent over method="GET", exposing sensitive credentials via the location bar.
- **Missing Accessibility & Validation:** Missing essential input labels, explicit element IDs, and validation markers (required).

2. Analyze Why the HTTP Request Is Not Triggered

- A standard <button> without a defined type inside a form defaults to type="submit". Explicitly declaring type="button" intentionally disables native form transmission logic.
- Without attached JavaScript event listeners calling .submit(), user clicks produce no operational network calls or payloads.

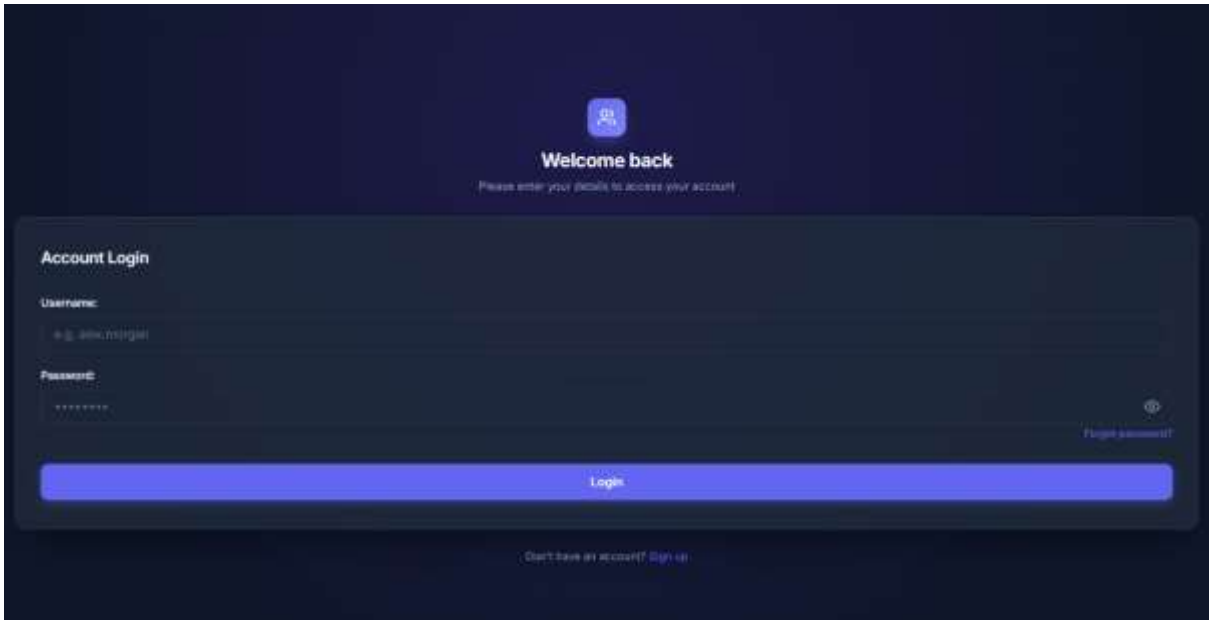
3. Interpret the Role of GET Method in This Scenario

Feature	Analysis & Risk Profile
Transmission Mechanism	Encodes input fields into the URI query string (e.g., /submit?username=admin&password=secret).
Security Risks	• Passwords stored in plain text inside browser histories, proxy records, and server logs. • Sensitive parameters exposed via HTTP Referer headers. • Non-compliant with HTTP standard definitions for safe and idempotent requests.

4. Propose a Corrected Implementation

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login</title>
</head>
<body>
  <main>
    <section>
      <h2>Account Login</h2>
      <form action="/submit" method="POST">
        <div>
          <label for="username">Username:</label>
          <input type="text" id="username" name="username" required
autocomplete="username">
        </div>
        <div>
          <label for="password">Password:</label>
          <input type="password" id="password" name="password" required
autocomplete="current-password">
        </div>
        <div>
          <button type="submit">Login</button>
        </div>
      </form>
    </section>
  </main>
</body>
</html>
```



5. Suggest Improvements for Secure Data Transmission

- **Use POST Method:** Transmit sensitive payload items encapsulated inside the HTTP message request body.
- **Mandate HTTPS (TLS/SSL):** Enforce encrypted transportation layers to neutralize eavesdropping and interception attacks.
- **Implement Anti-CSRF Tokens:** Include unique non-predictable form tokens to secure against Cross-Site Request Forgery.
- **Configure Security Headers:**
    - Strict-Transport-Security: max-age=31536000; includeSubDomains
    - Cache-Control: no-store
    - X-Frame-Options: DENY